

به نام خدا



دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس امنیت سخت افزار تمرین اول

نام و نام خانوادگی	بهزاد جنتی
شماره دانشجویی	810103098
تاریخ ارسال گزارش	1404/02/03
استاد	دکتر سیامک محمدی
ترم	بهار 1404

فهرست

6	بخش اول: پیاده سازی الگوریتم present
6	1. شماتیک الگوریتم present
7	2. تولید کلید ها برای هر راند الگوریتم:
8	3. پیاده سازی sbbox:
9	سوال امتیازی:
9	4. پیاده سازی pbox (permutation box):
10	سوال امتیازی 2:
11	5. پیاده سازی present-encryption:
13	6. پیاده سازی present-decryption:
14	7. تست نهایی الگوریتم present پیاده سازی شده:
15	بخش دوم: محاسبه خطا برای مدهای مختلف
15	1. مد ECB یا electronic codebook
15	2. مد CBC یا cipher block chaining:
16	3. مد CTR یا counter mode:
17	4. تست هر سه مود از نظر خطا:
20	سوال 2: پیاده سازی یک RFID Tag Reader
22	1. پیاده سازی server.py
24	2. پیاده سازی client.py
26	3. تست برقراری ارتباط بین سرور و کلاینت
28	4. سناریو Wrong PSK
28	1. سناریو Replay Attack
30	2. حمله MITM:

شکل‌ها

- شکل 1- تصویر الگوریتم present 6
- شکل 2- کد بخش key round generation 7
- شکل 3- کد پیاده سازی sbox 8
- شکل 4- پیاده سازی pbox 10
- شکل 5- تابع تعریف شده برای addRoundKey 10
- شکل 6- پیاده سازی present encrypt 12
- شکل 7- خروجی present encrypt با فرض مقاله 12
- شکل 8- کد تابع present-decrypt 13
- شکل 9- تست صحت پیاده سازی الگوریتم 14
- شکل 10- پیاده سازی ECB-decrypt 15
- شکل 11- کد پیاده سازی CBC-decrypt 16
- شکل 12- پیاده سازی مد CTR-decrypt 17
- شکل 13- رمزگشایی در سه مد در حالت عادی 17
- شکل 14- مقادیر اولیه جهت انجام رمزگشایی در سه مد 18
- شکل 15- تزریق ارور در سایفرتکست 18
- شکل 16- خروجی ارور در هر سه مد 19
- شکل 17- کد ارزیابی نتایج 19
- شکل 18- پروتکل ارتباط در RFID 20
- شکل 19- ایجاد سوکت برای سرور 22
- شکل 20- توابع تعریف شده برای استفاده در پروتکل 22
- شکل 21- بخش اول ارتباط بین سرور و کلاینت 23
- شکل 22- دریافت مقادیر H2 و M2 از تگ 23
- شکل 23- تایید H2 و ارسال H3 برای تگ 24
- شکل 24- مقادیر ابتدایی برای پیاده سازی پروتکل 24
- شکل 25- اتصال تگ به سرور یا reader 24
- شکل 26- بررسی درستی H1 توسط تگ بعد از دریافت مقادیر از سرور 25
- شکل 27- ارسال H2 و M2 برای سرور 25
- شکل 28- اتمام فرآیند صحت سنجی ارتباط 26
- شکل 29- ران گرفتن فایل server برای شروع کار 26
- شکل 30- ران گرفتن فایل کلاینت برای اتمام فرآیند اتصال 27
- شکل 31- تصویر خروجی سرور بعد از صحت سنجی ارتباط 27

28	شکل 32- خروجی در wrong PSK
29	شکل 33- خروجی ترمینال سرور در Replay Attack
30	شکل 34- خروجی ترمینال کلاینت با پورت 65433 در MITM attack
31	شکل 35- خروجی MITM

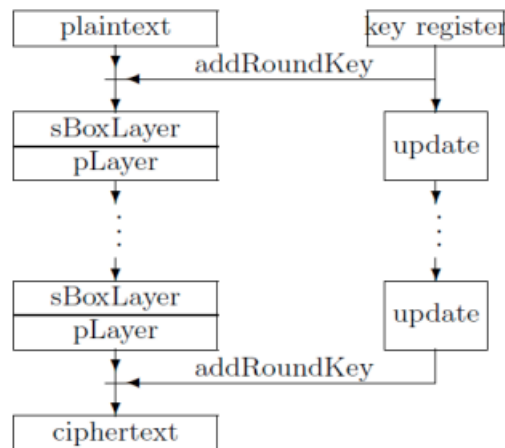
جدول‌ها

جدول 1- مقایسه سه مد در این سناریو در برابر خطا 20

بخش اول: پیاده سازی الگوریتم present

۱. شماتیک الگوریتم present

در تصویر زیر میتوانید مراحل پیاده سازی این الگوریتم که در مقاله present عنوان شده است را مشاهده کنید:



شکل 1- تصویر الگوریتم present

حال به توضیحاتی که در مورد کارکرد این الگوریتم در صورت سوال قرار داده شده است توجه میکنیم و طبق توضیحات به نوشتن کد پایتون برای پیاده سازی آن می‌پردازیم:

در این تمرین، به الگوریتم PRESENT با کلید 80 بیتی می‌پردازیم. در ساختار key schedule اصلی (Master Key) 80 بیتی به 32 کلید دور (Round Key) 64 بیتی تبدیل خواهد شد. ساختار key schedule به شرح زیر است:

1. برای کلید اولین دور، 64 بیت سمت چپ از 80 بیت کلید اصلی به عنوان کلید دور اول انتخاب می‌شود.
2. در ادامه برای دور های بعد یک سری تغییرات روی 80 بیت کلید اصلی انجام می‌شود و بعد از اعمال تغییرات دوباره 64 بیت سمت چپ به عنوان کلید همان دور انتخاب می‌شود. این اپدیت روی 80 بیت به شکل زیر است:

- ابتدا 61 بیت Left rotation shift می‌دهیم.
- 4 بیت سمت چپ را به S-box اعمال می‌کنیم و جایگذاری می‌کنیم. (*)
- سپس حاصل را با عدد بانری شماره دور XOR می‌کنیم. (یعنی 5 بیت سمت راست با عدد بانری شماره دور XOR می‌شود).

بعد از اعمال این تغییرات، 64 بیت سمت چپ (most significant bits) برای کلید همان دور انتخاب می‌شود.

2. تولید کلید ها برای هر راند الگوریتم:

همان طور که در توضیحات صفحه قبل مشاهده کردید در این الگوریتم 32 راند داریم که در هر راند خروجی ما تا آن مرحله (در ابتدای کار plaintext) با این کلید bitwise xor میشود.خب برای اولین دور کافیهست که 64 بیت سمت چپ از 80 بیت کلید اصلی (master) رو با متن plaintext بیایم xor کنیم. در کد زیر میتوانید پیاده سازی این بخش رو مشاهده کنید:

```
[ ] def generate_round_keys(master_key):
    """
    Generate 32 round keys from the 80-bit master key.

    Args:
        master_key: 80-bit integer (the master key)

    Returns:
        List of 32 64-bit integers (the round keys)
    """
    # Initialize key register with the master key
    key = master_key

    # Initialize list to store round keys
    round_keys = []

    # Generate 32 round keys
    for i in range(1, 33):
        # Extract the leftmost 64 bits as the round key
        round_keys.append((key >> 16) & 0xFFFFFFFFFFFFFFF)

        # Rotate key register by 61 positions to the left
        key = ((key & 0xFFFFFFFFFFFFFFF) << 61) | (key >> 19)
        key &= 0xFFFFFFFFFFFFFFF # Ensure it's still 80 bits

        # Pass the leftmost 4 bits through the S-box
        left_4_bits = (key >> 76) & 0xF
        key &= ~(0xF << 76) # Clear the leftmost 4 bits
        key |= SBOX[left_4_bits] << 76 # Replace with S-box output

        # XOR bits 19-15 with the round counter (paper)
        key ^= (i & 0x1F) << 15
        # c. XOR round counter with rightmost 5 bits (assignment specification)
        # key_register ^= (round_counter & 0x1F)
    return round_keys
```

شکل 2- کد بخش key round generation

در ادامه برای راند های بعدی باید کلید هر بار به شکل زیر آپدیت شود:

1. 61 بیت کلید به سمت چپ شیفت داده میشود.
 2. 4 بیت سمت چپ کلید به یک sbbox وارد شده و جایگزین میشود.
 3. در این بخش تفاوت در پیاده سازی مقاله با تمرین وجود دارد:
- مقاله بیت های 15 تا 19 کلید رو با شماره دور XOR می کند ولی در تمرین گفته شده که پنج بیت سمت راست رو در نظر بگیرید ، هر دوی این پیاده سازی ها در این تمرین انجام داده شده است.

3. پیاده سازی sbbox:

در این بخش از کد به پیاده سازی sbbox طبق شکل داده شده توسط سوال می پردازیم:

S-box های PRESENT ، 4 بیت به 4 بیت می باشند که جدول نگاشت آن ها به شکل زیر است:

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S[x]$	C	5	6	B	9	0	A	D	3	E	F	8	4	7	1	2

حال در زیر میتوانید پیاده سازی این بخش رو در کد مشاهده کنید ، از این sbbox ها در این الگوریتم با اسم sbbox-layer هم استفاده شده که میاد و 4 بیت و 4 بیت ورودی رو به این sbbox میده و خروجی تولید میشه.

```
[ ] # PRESENT S-box (4-bit to 4-bit)
# S-box in hexadecimal notation
SBOX = [0xC, 0x5, 0x6, 0xB, 0x9, 0x0, 0xA, 0xD, 0x3, 0xE, 0xF, 0x8, 0x4, 0x7, 0x1, 0x2]

def sbbox_layer(state):
    """
    Apply the S-box layer to the state.

    Args:
        state: 64-bit integer (the current state)

    Returns:
        64-bit integer (the state after S-box application)
    """
    result = 0

    # Apply S-box to each 4-bit nibble
    for i in range(16):
        nibble = (state >> (4 * i)) & 0xF
        result |= SBOX[nibble] << (4 * i)

    return result
```

شکل 3- کد پیاده سازی sbbox

سوال امتیازی:

سوال امتیازی: علت استفاده از S-box در (*) چیست؟ (5 نمره)

این مورد در فایل جویتر که به این گزارش الحاق شده به شکل کامل توضیح داده شده است. به شکل کلی sbox یک روش کریپتوگرافی یا encryption از نوع substitution است به این معنی که چند بیت از ورودی رو با چند بیت دیگه عوض میکنه. حالا مزیت این در این الگوریتم چیه؟

1. Diffusion: یعنی تاثیر بیت های plaintext رو پخش کنیم و تغییر یک بیت در متن اصلی باعث تغییر چند بیت در متن رمزنگاری شده(ciphertext) میشود. در اینجا البته منظور این است که تغییر کوچک در ورودی به خاطر اینکه این لایه با لایه permutation استفاده میشود که بعدا به آن میپردازیم باعث تغییرات غیر قابل پیش بینی در خروجی میشود و خب از این نظر باعث پیچیده شدن الگوریتم ما و همچنین سخت تر شدن حمله به آن میشود.

2. Confusion: هر بیت رمز نگاری شده تاثیر گرفته از چند بیت کلید باشد، که وقتی در هر راند میایم و کلید رو آپدیت میکنیم و بعد با متن ایکس اور میکنیم این اتفاق می افتد. 3. غیر خطی بودن: این بخش جزو بخش های غیر خطی الگوریتم ما بوده و باعث میشود تا حمله کردن به این الگوریتم دشوار شود ، در اینجا هم چه در بخش آپدیت کلید و چه در خود الگوریتم این مزیت را برای ما فراهم میکند.

4. پیاده سازی pbox (permutation box):

و در permutation layer جایگشت ها روی بیت ها انجام می شود که به شکل زیر است:

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$P(i)$	0	16	32	48	1	17	33	49	2	18	34	50	3	19	35	51
i	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
$P(i)$	4	20	36	52	5	21	37	53	6	22	38	54	7	23	39	55
i	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
$P(i)$	8	24	40	56	9	25	41	57	10	26	42	58	11	27	43	59
i	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
$P(i)$	12	28	44	60	13	29	45	61	14	30	46	62	15	31	47	63

برای پیاده سازی این بخش ابتدا یک آرایه 64 خانه ای میسازیم و مقادیر در هر اندیس آرایه رو طبق تصویر داده شده مشخص میکنیم.

از این pbox جهت تعریف player هم استفاده کردیم که در کد زیر میتوانید مشاهده کنید، این لایه به این صورت است که بیت به بیت ورودی بهش وارد میشه و اون بیت ورودی با توجه به اینکه بیت چندم است با یک بیت ثابت جا به جا (permutation یا جایگشت) میشود.

```
[ ] # Permutation table - maps bit i to position P(i)
# Permutation layer
P_LAYER = [0, 16, 32, 48, 1, 17, 33, 49, 2, 18, 34, 50, 3, 19, 35, 51,
            4, 20, 36, 52, 5, 21, 37, 53, 6, 22, 38, 54, 7, 23, 39, 55,
            8, 24, 40, 56, 9, 25, 41, 57, 10, 26, 42, 58, 11, 27, 43, 59,
            12, 28, 44, 60, 13, 29, 45, 61, 14, 30, 46, 62, 15, 31, 47, 63]

def perm_layer(state):
    """
    Apply the permutation layer to the state.

    Args:
        state: 64-bit integer (the current state)

    Returns:
        64-bit integer (the state after permutation)
    """
    result = 0

    # Move each bit according to the permutation table
    for i in range(64):
        if (state & (1 << i)) != 0:
            result |= 1 << P_LAYER[i]

    return result
```

شکل 4- پیاده سازی pbox

پیاده سازی addRoundKey:

همان طور که پیشتر گفته شد در هر راند باید ابتدا استیت مون در کلید راند ایکس اور شود ، همچنین در دور آخر فقط همین یک بخش رو داریم.

```
[ ] def add_round_key(state, round_key):
    """XOR state with round key"""
    return state ^ round_key
```

شکل 5- تابع تعریف شده برای addRoundKey

سوال امتیازی 2:

تمامی این عملیات ها برای 31 دور انجام می شود و در دور 32 ام فقط کلید دور آخر XOR می شود و ciphertext بدست می آید.

سوال امتیازی: چرا در دور نهایی فقط AddRoundKey اعمال می شود. (5 نمره)

خب جواب این سوال هم در فایل جویتر به شکل کامل داده شده است ولی در اینجا به شکل مختصر به آن اشاره میکنیم:

1. این بخش باعث میشود تا مهاجم نتواند به شکل برگشتی الگوریتم را پیشمایش کند و آن را بشکند و این باعث میشود در برابر حملات مقاوم تر باشیم.
 2. نبود دو لایه دیگه در بخش آخر باعث میشه سر بار کمتری داشته باشیم.
 3. اگه از این بخش استفاده نمیکردیم باعث میشد اون بخش آخر به شکل یک bottleneck در بیاد که در برابر حملات مقاوم نبود. (سایر بخش ها مثل sbox و pbox تبدیلات ثابتی هستند و در برابر حمله ممکن است مقاوم نباشند پس استفاده به تنهایی از این مورد در بخش آخر مطلوب است).
- سایر توضیحات در فایل جویتر داده شده است.

5. پیاده سازی present-encryption:

برای پیاده سازی این بخش فقط کافیه از توابع ساخته شده در بخش های قبل استفاده کنیم و طبق شماتیک الگوریتم این بخش رو پیاده کنیم ، این بخش همان طور که از اسمش پیداست جهت رمزنگاری متن ابتدایی به کار میرود.

با استفاده از توضیحات داده شده و مقاله ی PRESENT ، در ابتدا با پیاده سازی الگوریتم با پایتون بدون استفاده از توابع آماده ی الگوریتم PRESENT ، بلوک 64 بیتی (8 بیتی) زیر را با master key مربوط رمز کنید و سپس با رمزگشایی آن به متن plaintext اول برسید. (20 نمره)

Plaintext: 0xC2F824E0A5DC2963

Master Key: 0x6CF590CEE432114A19F4

برای تست این بخش از کلید به همراه plaintext داده شده در متن صورت سوال استفاده میکنیم.

کد مربوط به این بخش در صفحه بعد قرار داده شده است.

```
[ ] def present_encrypt(plaintext, master_key):
    """
    Encrypt a 64-bit block using PRESENT with an 80-bit key.

    Args:
        plaintext: 64-bit integer (the plaintext block)
        master_key: 80-bit integer (the master key)

    Returns:
        64-bit integer (the ciphertext block)
    """
    # Initialize state with plaintext
    state = plaintext

    # Generate round keys
    round_keys = generate_round_keys(master_key)

    # 31 rounds of processing
    for i in range(31):
        # Add round key
        state = state ^ round_keys[i]

        # S-box layer
        state = sbox_layer(state)

        # Permutation layer
        state = perm_layer(state)

    # Final key addition (post-whitening)
    state = state ^ round_keys[31]

    return state
```

شکل 6- پیاده سازی `present_encrypt`

خب در ادامه می‌توانید خروجی تست با ورودی های سوال رو مشاهده کنید:

```
[ ] # Test values from assignment
plaintext = 0xC2F824E0A5DC2963
master_key = 0x6CF590CEE432114A19F4

# Encrypt
ciphertext = present_encrypt(plaintext, master_key)
print(f"Plaintext: {hex(plaintext)}")
print(f"Master Key: {hex(master_key)}")
print(f"Ciphertext: {hex(ciphertext)}")
```

```
⇒ Plaintext: 0xc2f824e0a5dc2963
   Master Key: 0x6cf590cee432114a19f4
   Ciphertext: 0x5f4a2f899db6db86
```

شکل 7- خروجی `present_encrypt` با فرض مقاله

نکته مهم: با توجه به اینکه سوال در بخش آخر آپدیت کلید برای هر راند فرض متفاوتی با مقاله گرفته است برای این بخش هم دو تا جواب موجود است که هر دو در فایل جویپتر قرار داده شده است.

6. پیاده سازی present-decryption:

پیاده سازی این بخش هم به مانند قبل است با این فرق که باید تمام المان هایی که ساخته بودیم را معکوس کنیم مثل inverse sbox و inverse pbox.

برای جلوگیری از طولانی شدن گزارش این کدها (معکوس کردن توابع قبلی) قرار داده نشده ولی در فایل جویپتر قابل رویت است.

حال در زیر میتوانید کد مربوط به این بخش را مشاهده کنید.

```
[ ] def present_decrypt(ciphertext, master_key):
    """
    Decrypt a 64-bit block using PRESENT with an 80-bit key.

    Args:
        ciphertext: 64-bit integer (the ciphertext block)
        master_key: 80-bit integer (the master key)

    Returns:
        64-bit integer (the plaintext block)
    """
    # Initialize state with ciphertext
    state = ciphertext

    # Generate round keys
    round_keys = generate_round_keys(master_key)

    # Final key addition (post-whitening)
    state = state ^ round_keys[31]

    # 31 rounds of processing (in reverse)
    for i in range(30, -1, -1):
        # Inverse permutation layer
        state = inv_perm_layer(state)

        # Inverse S-box layer
        state = inv_sbox_layer(state)

        # Add round key
        state = state ^ round_keys[i]

    return state
```

شکل 8- کد تابع present-decrypt

7. تست نهایی الگوریتم present پیاده سازی شده:

در این بخش با رمز نگاری کردن plaintext و سپس رمزگشایی کردن ciphertext درستی پیاده سازی مون رو چک میکنیم.

```
[ ] # Main function to test the implementation
def main():
    # Part 1: Test PRESENT encryption/decryption
    plaintext = 0xC2F824E0A5DC2963
    master_key = 0x6CF590CEE432114A19F4

    print("Part 1: PRESENT Block Cipher Implementation")
    print("-----")
    print(f"Plaintext: 0x{plaintext:016X}")
    print(f"Master Key: 0x{master_key:020X}")

    # Encrypt
    ciphertext = present_encrypt(plaintext, master_key)
    print(f"Ciphertext: 0x{ciphertext:016X}")

    # Decrypt
    decrypted = present_decrypt(ciphertext, master_key)
    print(f"Decrypted: 0x{decrypted:016X}")

    # Verify
    assert decrypted == plaintext, "Decryption failed!"
    print("Encryption and decryption successful!")
    print()
```

```
⇒ Part 1: PRESENT Block Cipher Implementation
-----
Plaintext: 0xC2F824E0A5DC2963
Master Key: 0x6CF590CEE432114A19F4
Ciphertext: 0x5F4A2F899DB6DB86
Decrypted: 0xC2F824E0A5DC2963
Encryption and decryption successful!
```

شکل 9-تست صحت پیاده سازی الگوریتم

همان طور که در تصویر میبینید متن رمزگشایی شده با متنی که در ابتدا داشتیم یکی شد که این نشان میدهد که به درستی عمل رمزگذاری و رمزگشایی رو انجام داده ایم.

بخش دوم: محاسبه خطا برای مدهای مختلف

1. مد ECB یا electronic codebook

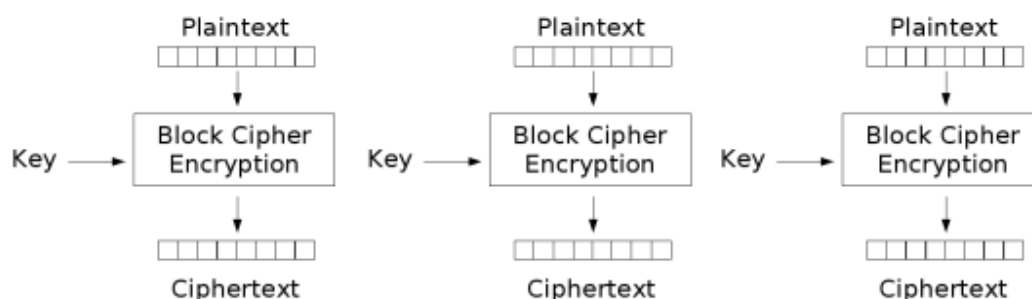
این مد به این شکل کار میکند هر بلوک از plaintext به شکل مجزا با کلید رمزگذاری میشود. میتوانی پیاده سازی این مد توسط present_decrypt رو در زیر مشاهده کنید:

Code Implementation

```
[ ] def ecb_decrypt(ciphertext, key, block_size=8):  
    """Decrypt using ECB mode"""  
    plaintext = bytearray()  
    for i in range(0, len(ciphertext), block_size):  
        block = int.from_bytes(ciphertext[i:i+block_size], 'big')  
        decrypted_block = present_decrypt(block, key)  
        plaintext.extend(decrypted_block.to_bytes(block_size, 'big'))  
    return plaintext
```

شکل 10- پیاده سازی ECB-decrypt

تصویری از نحوه عملکرد این مد نیز در زیر قرار داده شده است.



Electronic Codebook (ECB) mode encryption

2. مد CBC یا cipher block chaining

در این مد رمزنگاری هر بلاک به بلاک قبل وابسته است. یعنی قبل از اینکه یک بلاک از plaintext توسط کلید رمزنگاری شود باید با ciphertext (خروجی بلاک رمزنگاری شده قبل) XOR شود. کد مربوط به این مد رو میتونید در صفحه بعد مشاهده کنید:

Code Implementation

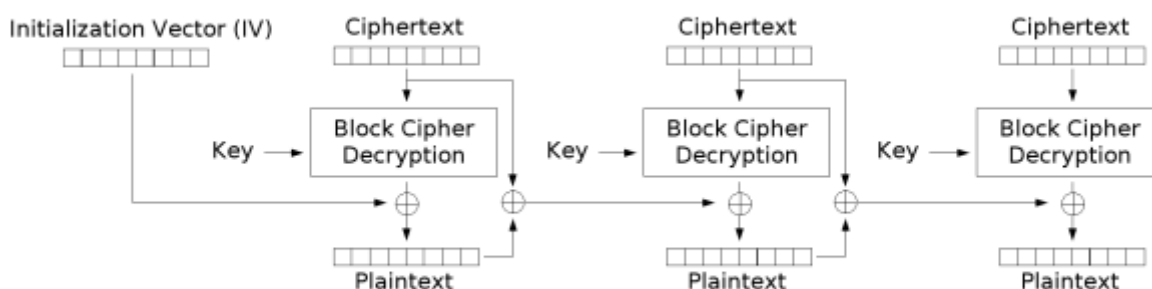
```
[ ] def cbc_decrypt(ciphertext, key, iv, block_size=8):
    """Decrypt using CBC mode"""
    plaintext = bytearray()
    prev_block = iv

    for i in range(0, len(ciphertext), block_size):
        block = int.from_bytes(ciphertext[i:i+block_size], 'big')
        decrypted_block = present_decrypt(block, key)
        plaintext_block = decrypted_block ^ prev_block
        plaintext.extend(plaintext_block.to_bytes(block_size, 'big'))
        prev_block = block

    return plaintext
```

شکل 11- کد پیاده سازی CBC-decrypt

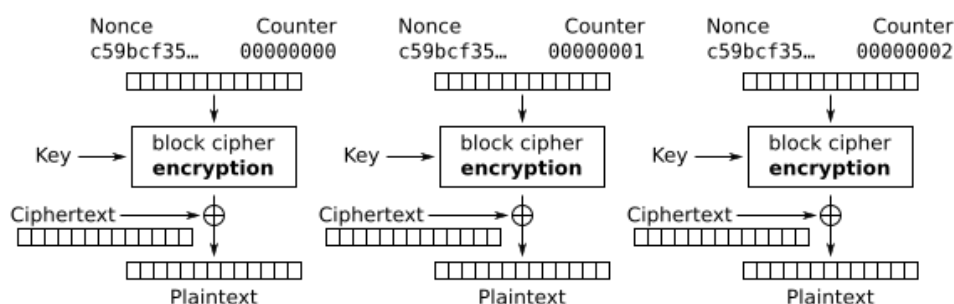
در زیر میتونید تصویر مربوط به این مد رو مشاهده کنید:



Propagating Cipher Block Chaining (PCBC) mode decryption

3. مد CTR یا counter mode:

این مد هر block cipher رو تبدیل به یک stream cipher میکند و سپس بایت به بایت شروع به رمز گشایی میکند. پیاده سازی مربوط به این مد در زیر میتونید مشاهده کنید:



Counter (CTR) mode decryption

در زیر پیاده سازی کد این مد رو هم میتونید مشاهده کنید:

Code Implementation

```
[ ] def ctr_decrypt(ciphertext, key, nonce, block_size=8):
    """Decrypt using CTR mode"""
    plaintext = bytearray()
    counter = 0

    for i in range(0, len(ciphertext), block_size):
        # Combine nonce and counter
        counter_block = (nonce << 32) | counter
        # Encrypt counter block
        keystream = present_encrypt(counter_block, key)
        # XOR with ciphertext
        ctext_block = int.from_bytes(ciphertext[i:i+block_size], 'big')
        ptext_block = ctext_block ^ keystream
        plaintext.extend(ptext_block.to_bytes(block_size, 'big'))
        counter += 1

    return plaintext
```

شکل 12- پیاده سازی مد CTR-decrypt

4. تست هر سه مود از نظر خطا:

در ابتدا طبق خواسته کتاب اومدیم و ciphertext که شامل سه سایفربلاک هست رو در هر سه مود decrypt کردیم.

در ضمن همان طور که در صورت سوال ذکر شده بود برای کامل کردن سایفرتکست از شماره دانشجویی خودم استفاده کردم.

```
[ ] # Decrypt using each mode
ecb_plaintext = ecb_decrypt(ciphertext_bytes, demo_key)
cbc_plaintext = cbc_decrypt(ciphertext_bytes, demo_key, demo_iv)
ctr_plaintext = ctr_decrypt(ciphertext_bytes, demo_key, demo_nonce)

print("Original decryption results:")
print(f"ECB: {ecb_plaintext.hex(' ')}")
print(f"CBC: {cbc_plaintext.hex(' ')}")
print(f"CTR: {ctr_plaintext.hex(' ')}")
```



Original decryption results:

ECB: ba e3 4d bd e6 65 b4 e8 ab 49 05 0b 0f 9d a1 8e f1 1d d4 0e 9b c7 1a 99
CBC: bb c0 08 da 6f ce 79 07 1b 6c a7 1b 19 04 5e 74 ab 85 2c a4 7f b8 f4 f1
CTR: b3 f8 26 42 52 fd 69 fb 6c 38 e1 e7 0e c0 bd 0f 4c a8 26 07 f4 a3 3e 1f

شکل 13- رمزگشایی در سه مد در حالت عادی

نکته: جاهایی که نیاز بود به متغیری مقدار اولیه داده شود از یک مقدار فرضی استفاده کردیم در زیر میتونید مقدار دهی ها رو مشاهده کنید، برای کلید هم از کلید بخش قبلی استفاده کردیم.

```
[ ] # Given ciphertext (with placeholders for x and y)
ciphertext_hex = "b0 25 a2 10 16 xx ff fa 5a xy f8 aa e4 7f ee 68 79 8b be a9 bb ff e3 30"

# Extract student ID digits
student_id = "810103098"
x = int(student_id[-2]) # Second-to-last digit
y = int(student_id[-1]) # Last digit

# Replace xx and xy in the ciphertext
ciphertext_hex = ciphertext_hex.replace("xx", f"{x}{x}").replace("xy", f"{x}{y}")

# Convert hex string to bytes
ciphertext_bytes = bytearray.fromhex(ciphertext_hex.replace(" ", ""))

# For demonstration, we'll use a fixed key and IV/nonce
demo_key = 0x6CF590CEE432114A19F4
demo_iv = 0x0123456789ABCDEF
demo_nonce = 0x01234567
```

شکل 14- مقادیر اولیه جهت انجام رمزگشایی در سه مد

حال میایم و همان طور که در سوال گفته شده یک بایت از بلاک اول سایفرتکست رو عوض میکنیم به نوعی ارور بهش تزریق میکنیم تا عملکرد هر سه مود رو مشاهده کنیم.

```
[ ] # Now introduce an error in the first block (change one byte)
corrupted_ciphertext = bytearray(ciphertext_bytes)
corrupted_ciphertext[2] ^= 0x01 # Flip one bit in the third byte
```

شکل 15- تزریق ارور در سایفرتکست

حال میایم و متن سایفرتکست خطا دار رو رمزگشایی میکنیم و به محاسبه ارور برای هر مد میپردازیم:

نکته: در صورت سوال این طور فرض شده که خطا زمانی در پیام رخ میدهد که فرستنده آن را رمزگذاری کرده و برای ارسال روی کانال قرار میدهد و به خاطر مثلا نویز کانال این ارور رخ میدهد.

```
[ ] # Decrypt the corrupted ciphertext
ecb_corrupted = ecb_decrypt(corrupted_ciphertext, demo_key)
cbc_corrupted = cbc_decrypt(corrupted_ciphertext, demo_key, demo_iv)
ctr_corrupted = ctr_decrypt(corrupted_ciphertext, demo_key, demo_nonce)

print("\nDecryption results after corruption:")
print(f"ECB corrupted: {ecb_corrupted.hex(' ')}")
print(f"CBC corrupted: {cbc_corrupted.hex(' ')}")
print(f"CTR corrupted: {ctr_corrupted.hex(' ')}")

# Count and identify which bytes are affected by the error
def count_affected_bytes(original, corrupted):
    affected = 0
    affected_positions = []
    for i in range(len(original)):
        if original[i] != corrupted[i]:
            affected += 1
            affected_positions.append(i)
    return affected, affected_positions

ecb_affected, ecb_positions = count_affected_bytes(ecb_plaintext, ecb_corrupted)
cbc_affected, cbc_positions = count_affected_bytes(cbc_plaintext, cbc_corrupted)
ctr_affected, ctr_positions = count_affected_bytes(ctr_plaintext, ctr_corrupted)

print("\nError propagation analysis:")
print(f"ECB: {ecb_affected} bytes affected at positions {ecb_positions}")
print(f"CBC: {cbc_affected} bytes affected at positions {cbc_positions}")
print(f"CTR: {ctr_affected} bytes affected at positions {ctr_positions}")
```

شکل 17- کد ارزیابی نتایج

حال طبق کد بالا و ورودی های مسئله و مقادیر پیش فرضی که در نظر گرفتیم به خروجی زیر

رسیدیم:



```
Decryption results after corruption:
ECB corrupted: 4b 9f c7 05 3d 27 65 0e ab 49 05 0b 0f 9d a1 8e f1 1d d4 0e 9b c7 1a 99
CBC corrupted: 4a bc 82 62 b4 8c a8 e1 1b 6c a6 1b 19 04 5e 74 ab 85 2c a4 7f b8 f4 f1
CTR corrupted: b3 f8 27 42 52 fd 69 fb 6c 38 e1 e7 0e c0 bd 0f 4c a8 26 07 f4 a3 3e 1f

Error propagation analysis:
ECB: 8 bytes affected at positions [0, 1, 2, 3, 4, 5, 6, 7]
CBC: 9 bytes affected at positions [0, 1, 2, 3, 4, 5, 6, 7, 10]
CTR: 1 bytes affected at positions [2]
```

شکل 16- خروجی ارور در هر سه مد

همان طور که مشاهده میکنید در مد اول 8 بایت و در مد دوم 9 بایت و در مد آخر تنها یک

بایت تحت تاثیر قرار گرفته اند.

حال در صفحه بعد اومدیم و تمام این موارد رو در یک جدول برای مقایسه قرار دادیم:

جدول مقایسه 3 مود از نظر تحمل پذیری در برابر خطا:

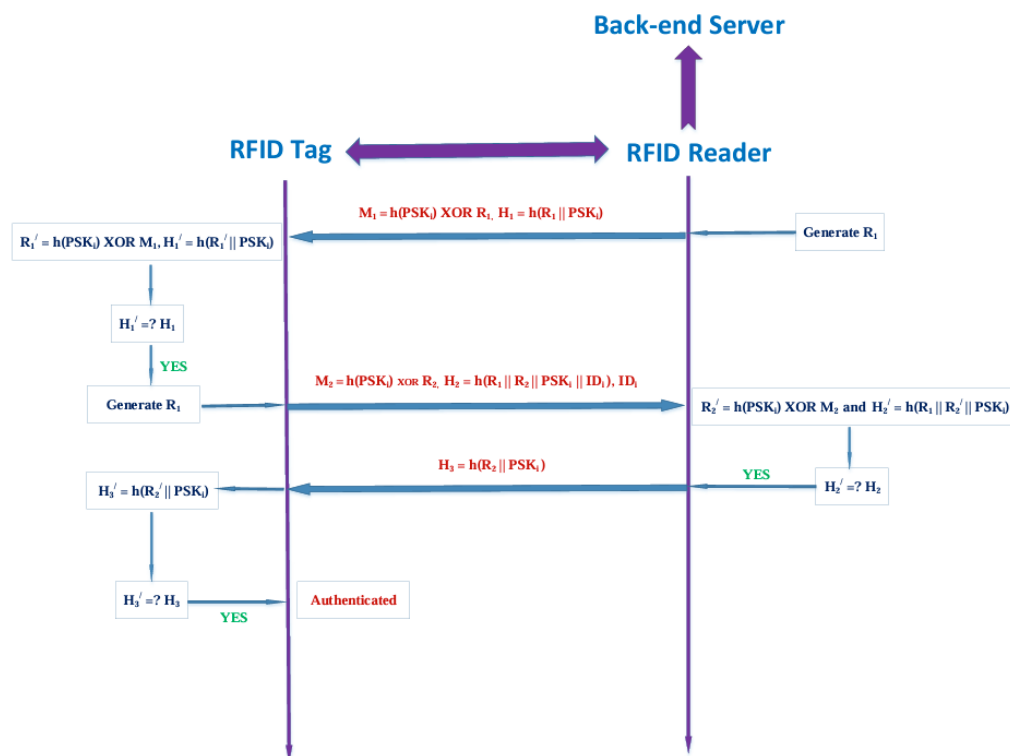
جدول 1- مقایسه سه مد در این سناریو در برابر خطا

ارزیابی کلی	تعداد بایت تاثیر گرفته از ارور	نام مدها
چون یک به یک است هیچ Diffusion ندارد و ضعیف در برابر ارور	8	ECB
مقاوم ولی ضعیف در برابر خطا	9	CBC
تحمل پذیری بالا در برابر خطا	1	CTR

سوال 2: پیاده سازی یک RFID Tag Reader

در این بخش از ما خواسته شده تا طبق خواسته صورت سوال و مقاله A Lightweight Protocol for RFID Authentication یک پروتکل RFID پیاده سازی کنیم که از دو بخش RFID reader و RFID tag تشکیل شده است.

کلیت مراحل برقراری ارتباط بین سرور (RFID reader) و کلاینت (RFID tag) در مقاله توضیح داده شده است و میتوانید در تصویر زیر نیز مشاهده کنید:



2. Proposed mutual authentication protocol between RFID tag and reader.

شکل 18- پروتکل ارتباط در RFID

در این پروژه، ما پروتکل احراز هویت دوطرفه RFID سبک‌وزن را که در مقاله Ray و Azad معرفی شده، پیاده‌سازی کردیم. هدف اصلی، برقراری ارتباط امن بین یک تگ RFID (کلاینت) و یک خواننده RFID (سرور) است. با توجه به محدودیت‌های منابع تگ‌های RFID، این پروتکل از عملیات‌های سبک‌وزن مانند توابع هاش، عملیات XOR و تولید اعداد تصادفی بهره می‌برد.

ابتدا با بررسی دقیق مقاله و الزامات پروژه، متوجه شدم که باید شماره دانشجویی خودم (810103098) را به عنوان کلید پیش‌اشتراکی (PSK) استفاده کنم. همچنین از آنجایی که شماره دانشجویی من زوج است، طبق دستورالعمل باید از الگوریتم هاش SHA-256 استفاده کنم. سپس به طراحی دو برنامه پرداختم: یکی برای سرور (خواننده RFID) و دیگری برای کلاینت (تگ RFID).

در سمت سرور، اولین گام ایجاد یک سوکت شبکه بود که به پورت مشخصی گوش می‌دهد. سرور پس از برقراری ارتباط، یک عدد تصادفی $R1$ تولید می‌کند و با استفاده از PSK، مقادیر $M1 = h(R1 \text{ PSK}) \text{ XOR } R1$ را محاسبه می‌نماید. این مقادیر به کلاینت ارسال می‌شوند. سپس سرور منتظر دریافت پاسخ از کلاینت می‌ماند.

در سمت کلاینت، برنامه به سرور متصل می‌شود و پس از دریافت $M1$ و $H1$ ، ابتدا با استفاده از عملیات $R1' = h(PSK) \text{ XOR } M1$ مقدار $R1'$ را بازیابی می‌کند. سپس با محاسبه $H1' = h(R1' \text{ PSK})$ و مقایسه آن با $H1$ دریافتی، صحت پیام را بررسی می‌کند. در صورت تایید، کلاینت یک عدد تصادفی $R2$ تولید کرده و مقادیر $M2 = h(PSK) \text{ XOR } R2$ و $H2 = h(R1' \text{ R2 PSK ID})$ را محاسبه و همراه با شناسه خود (ID) به سرور ارسال می‌کند.

سرور پس از دریافت پیام کلاینت، ابتدا $R2'$ را با استفاده از $M2$ بازیابی کرده و سپس $H2' = h(R1 \text{ R2'} \text{ PSK ID})$ را محاسبه می‌کند. اگر $H2'$ با $H2$ دریافتی برابر باشد، هویت کلاینت تایید شده و سرور مقدار $H3 = h(R2' \text{ PSK})$ را محاسبه و به کلاینت ارسال می‌کند.

در نهایت، کلاینت نیز $H3' = h(R2 \text{ PSK})$ را محاسبه کرده و با مقایسه آن با $H3$ دریافتی، هویت سرور را تایید می‌کند. به این ترتیب، فرآیند احراز هویت دوطرفه با موفقیت انجام می‌شود. برای تست کارایی پروتکل، سه سناریوی شکست نیز پیاده‌سازی کردم:

کلید نادرست: در این سناریو، کلاینت از یک PSK متفاوت استفاده می‌کند که باعث شکست در احراز هویت می‌شود.

حمله بازپخش: در این سناریو، کلاینت سعی می‌کند از پیام‌های قبلی ضبط شده استفاده کند، اما با توجه به استفاده از اعداد تصادفی جدید در هر نشست، این حمله شکست می‌خورد.

حمله مرد میانی: در این سناریو، پیام‌ها در حین انتقال تغییر داده می‌شوند که توسط مکانیزم یکپارچگی پیام (با استفاده از توابع هش) تشخیص داده می‌شود.

1. پیاده سازی server.py

برای راه اندازی سرور همان طور که در فایل صورت سوال آمده است باید از طریق لایبرری socket در پایتون بیایم و یه سوکت ایجاد کنیم و از طریق آن را به یک IP و port وصل میکنم.

```
def start_server():  
    """Start the RFID reader server"""  
    print("\033[92m[+] RFID reader listening on {}:{}\033[0m".format(HOST, PORT))  
    logging.info("RFID reader started")  
  
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:  
        s.bind((HOST, PORT))  
        s.listen()
```

شکل 19- ایجاد سوکت برای سرور

همچنین دو تابع برای استفاده در ادامه پروتکل پیاده سازی کردیم که یکی عدد رندوم تولید میکند و یکی دیگر هم تابع هش است:

```
def generate_random_number():  
    """Generate a random number for authentication"""  
    return random.randint(1000000, 9999999)  
  
def calculate_hash(data):  
    """Calculate SHA-256 hash of the data"""  
    if isinstance(data, str):  
        data = data.encode()  
    return hashlib.sha256(data).hexdigest()
```

شکل 20- توابع تعریف شده برای استفاده در پروتکل

نکته از جایی که شماره دانشجویی من زوج است طبق گفته سوال از sha256 استفاده کردیم.

در ادامه طبق شکل صفحه قبل سرور یک عدد رندوم برای R1 تولید میکند و سپس مقادیر M1 و H1 را تولید کرده و برای tag میفرستد.

تصویر مربوط به این بخش را میتوانید در صفحه بعد مشاهده کنید.

```

# Step 1: Generate R1
R1 = generate_random_number()
print(f"\033[92m[*] Generated random number R1: {R1}\033[0m")
logging.info(f"Generated random number R1: {R1}")

# Calculate h(PSK)
h_psk = calculate_hash(PSK)
logging.info(f"h(PSK): {h_psk}")

# Calculate M1 = h(PSK) XOR R1
R1_hex = format(R1, 'x')
M1 = xor_strings(h_psk, R1_hex)
logging.info(f"M1 = h(PSK) XOR R1: {M1}")

# Calculate H1 = h(R1 || PSK)
H1 = calculate_hash(str(R1) + PSK)
logging.info(f"H1 = h(R1 || PSK): {H1}")

# Step 2: Send M1 and H1 to tag
print("\033[92m[*] Sending authentication challenge to tag...\033[0m")
conn.sendall(f"{M1},{H1}".encode())

```

شکل 21- بخش اول ارتباط بین سرور و کلاینت

در ادامه سرور منتظر پاسخ تگ یا همان کلاینت می‌ماند و وقتی مقادیر M2 و H2 را از تگ دریافت کرد می‌آید و R2 رو محاسبه کرده و درست بودن H2 رو بررسی میکند. در صورت تایید در این مرحله H3 را تولید کرده و برای تگ می‌فرستد.

```

# Step 5: Receive M2 and H2 from tag
print("\033[92m[*] Waiting for tag response...\033[0m")
data = conn.recv(1024).decode()
if not data:
    print("\033[91m[-] No response received from tag\033[0m")
    logging.error("No response received from tag")
    continue

M2, H2 = data.split(',')
logging.info(f"Received M2: {M2}, H2: {H2}")
print("\033[92m[+] Received response from tag\033[0m")

```

شکل 22- دریافت مقادیر H2 و M2 از تگ

در صفحه بعد میتونید بخش آخر کار سرور رو هم مشاهده کنید.


```

# Step 6: Calculate R2 = h(PSK) XOR M2
R2_hex = xor_strings(h_psk, M2)
# Convert R2 from hex to int for further processing
try:
    R2 = int(R2_hex, 16)
    logging.info(f"Calculated R2: {R2}")

    # Verify H2 = h(R1 || R2 || PSK || ID)
    # In this case, ID is the same as PSK for simplicity
    H2_calculated = calculate_hash(str(R1) + str(R2) + PSK + PSK)
    logging.info(f"Calculated H2: {H2_calculated}")

    if H2 == H2_calculated:
        print("\033[92m[+] Tag authenticated successfully!\033[0m")
        logging.info("Tag authenticated successfully")

        # Step 7: Calculate H3 = h(R2 || PSK)
        H3 = calculate_hash(str(R2) + PSK)
        logging.info(f"H3 = h(R2 || PSK): {H3}")

        # Step 8: Send H3 to tag
        print("\033[92m[*] Sending confirmation to tag...\033[0m")
        conn.sendall(H3.encode())
        print("\033[92m[+] Mutual authentication completed successfully!\033[0m")
        logging.info("Mutual authentication completed")

```

شکل 23- تایید H2 و ارسال H3 برای تگ

2. پیاده سازی client.py

در ابتدا مقادیر آی دی و پورت و psk که بین سرور و کلاینت مشترک است در زیر قرار داده شده.

```

# Configuration
HOST = '127.0.0.1'
PORT = 65432
PSK = "810103098" # Pre-shared key (student ID)

```

شکل 24- مقادیر ابتدایی برای پیاده سازی پروتکل

در زیر نیز میتونید اتصال تگ به سرور رو مشاهده کنید:

```

def connect_to_reader():
    """Connect to the RFID reader and perform authentication"""
    print("\033[94m[*] RFID tag initializing...\033[0m")
    logging.info("RFID tag initialized")

    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
            print(f"\033[94m[*] Connecting to RFID reader at {HOST}:{PORT}...\033[0m")
            s.connect((HOST, PORT))
            print("\033[94m[+] Connected to RFID reader\033[0m")
            logging.info(f"Connected to RFID reader at {HOST}:{PORT}")

```

شکل 25- اتصال تگ به سرور یا reader

در ادامه باید تگ منتظر پیام سرور یا همان RFID reader بماند. بعد از دریافت M1 و H1 از سرور حال باید بیاد و R1 و H1 (در شکل `R1`, `H1`) رو محاسبه کرده و سپس درستی H1 رو بررسی کند و پس از تایید تگ یک عدد رندوم R2 تولید میکند و سپس بعد از محاسبه M2 و H2 آن ها را برای سرور ارسال کند.

```
# Step 3: Receive M1 and H1 from reader
data = s.recv(1024).decode()
M1, H1 = data.split(',')
print("\033[94m[+] Received authentication challenge from reader\033[0m")
logging.info(f"Received M1: {M1}, H1: {H1}")

# Calculate h(PSK)
h_psk = calculate_hash(PSK)
logging.info(f"h(PSK): {h_psk}")

# Calculate R1 = h(PSK) XOR M1
R1_hex = xor_strings(h_psk, M1)
# Convert R1 from hex to int for further processing
try:
    R1 = int(R1_hex, 16)
    logging.info(f"Calculated R1: {R1}")

    # Verify H1 = h(R1 || PSK)
    H1_calculated = calculate_hash(str(R1) + PSK)
    logging.info(f"Calculated H1: {H1_calculated}")

    if H1 == H1_calculated:
        print("\033[94m[+] Reader challenge verified successfully\033[0m")
        logging.info("Reader challenge verified successfully")
```

شکل 26- بررسی درستی H1 توسط تگ بعد از دریافت مقادیر از سرور

در ادامه هم بقیه کار رو مشاهده میکنید که پیش تر درباره آن صحبت شد.

```
# Step 4: Generate R2
R2 = generate_random_number()
print(f"\033[94m[*] Generated random number R2: {R2}\033[0m")
logging.info(f"Generated random number R2: {R2}")

# Calculate M2 = h(PSK) XOR R2
R2_hex = format(R2, 'x')
M2 = xor_strings(h_psk, R2_hex)
logging.info(f"M2 = h(PSK) XOR R2: {M2}")

# Calculate H2 = h(R1 || R2 || PSK || ID)
# In this case, ID is the same as PSK for simplicity
H2 = calculate_hash(str(R1) + str(R2) + PSK + PSK)
logging.info(f"H2 = h(R1 || R2 || PSK || ID): {H2}")

# Step 5: Send M2 and H2 to reader
print("\033[94m[*] Sending response to reader...\033[0m")
s.sendall(f"{M2},{H2}".encode())
```

شکل 27- ارسال H2 و M2 برای سرور

در این بخش تگ منتظر پیام سرور می‌ماند و بعد از دریافت H3 در آخرین مرحله درستی آن را سنجیده و Authentication اتمام می‌پذیرد.

```
# Step 9: Receive H3 from reader
print("\033[94m[*] Waiting for reader confirmation...\033[0m")
H3_received = s.recv(1024).decode()

if H3_received == "Authentication Failed":
    print("\033[91m[-] Authentication failed at reader side\033[0m")
    logging.error("Authentication failed at reader side")
    return

logging.info(f"Received H3: {H3_received}")

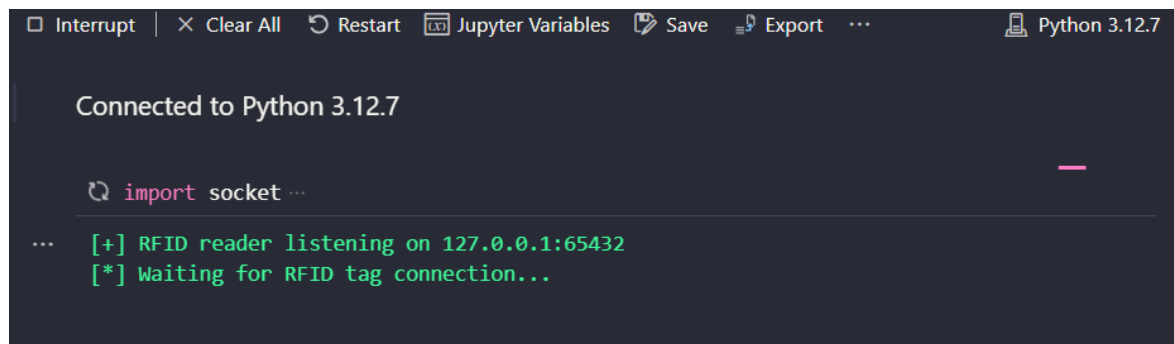
# Calculate expected H3 = h(R2 || PSK)
H3_expected = calculate_hash(str(R2) + PSK)
logging.info(f"Expected H3: {H3_expected}")

if H3_received == H3_expected:
    print("\033[94m[+] Reader authenticated successfully!\033[0m")
    print("\033[94m[+] Mutual authentication completed successfully!\033[0m")
    logging.info("Reader authenticated successfully")
    logging.info("Mutual authentication completed")
```

شکل 28- اتمام فرآیند صحت سنجی ارتباط

3. تست برقراری ارتباط بین سرور و کلاینت

ابتدا فایل server_RFID.py رو در یک ترمینال جدا اجرا میکنیم:



```
Interrupt | X Clear All | Restart | Jupyter Variables | Save | Export | Python 3.12.7

Connected to Python 3.12.7

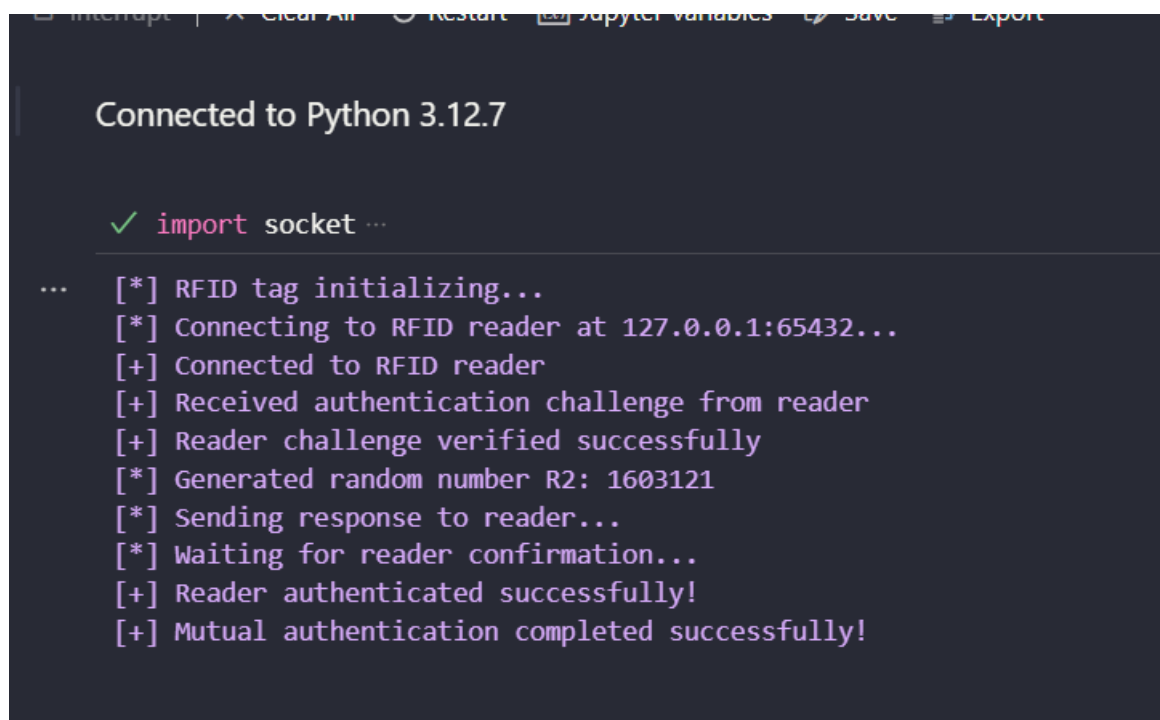
import socket ...

... [+] RFID reader listening on 127.0.0.1:65432
[*] Waiting for RFID tag connection...
```

شکل 29- ران گرفتن فایل server برای شروع کار

حال فایل client_RFID رو ران می‌گیریم.

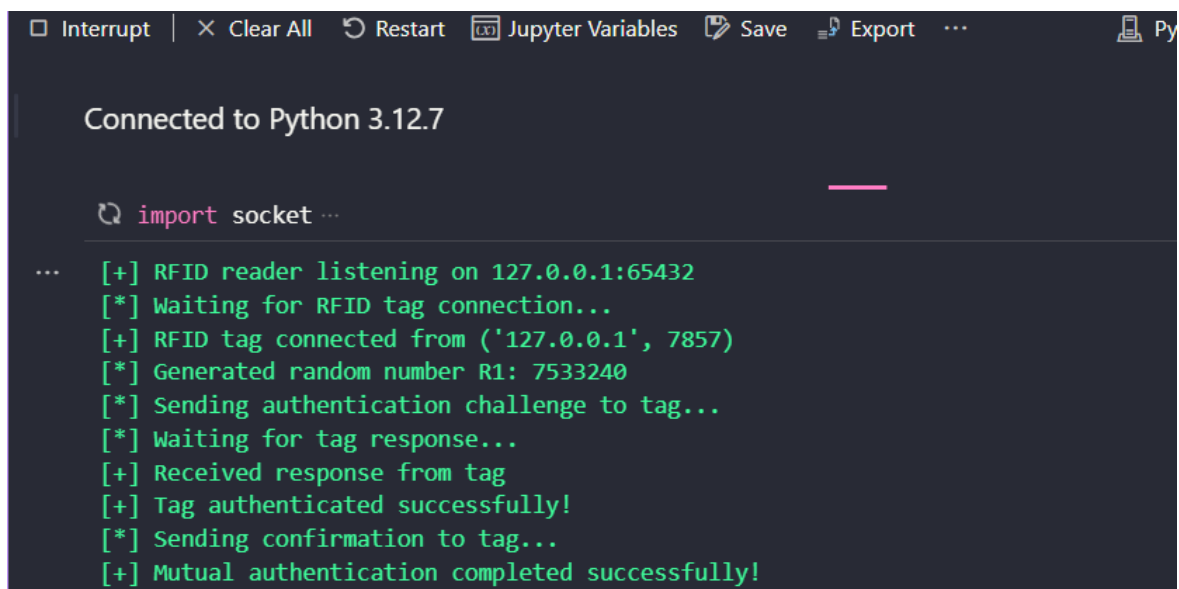
بعد از این ران اتصال برقرار میشود و تصویر این مورد رو در صفحه بعد میتونید مشاهده کنید:



```
Interrupt | X Clear All | Restart | Jupyter Variables | Save | Export ...  
  
Connected to Python 3.12.7  
  
✓ import socket ...  
... [*] RFID tag initializing...  
[*] Connecting to RFID reader at 127.0.0.1:65432...  
[+] Connected to RFID reader  
[+] Received authentication challenge from reader  
[+] Reader challenge verified successfully  
[*] Generated random number R2: 1603121  
[*] Sending response to reader...  
[*] Waiting for reader confirmation...  
[+] Reader authenticated successfully!  
[+] Mutual authentication completed successfully!
```

شکل 30- ران گرفتن فایل کلاینت برای اتمام فرآیند اتصال

در ادامه تصویر خروجی سرور هم گذاشته شده که نشان میدهد اتصال از طرف سرور هم تایید شده است:



```
Interrupt | X Clear All | Restart | Jupyter Variables | Save | Export ... | Py  
  
Connected to Python 3.12.7  
  
import socket ...  
... [+] RFID reader listening on 127.0.0.1:65432  
[*] Waiting for RFID tag connection...  
[+] RFID tag connected from ('127.0.0.1', 7857)  
[*] Generated random number R1: 7533240  
[*] Sending authentication challenge to tag...  
[*] Waiting for tag response...  
[+] Received response from tag  
[+] Tag authenticated successfully!  
[*] Sending confirmation to tag...  
[+] Mutual authentication completed successfully!
```

شکل 31- تصویر خروجی سرور بعد از صحت سنجی ارتباط

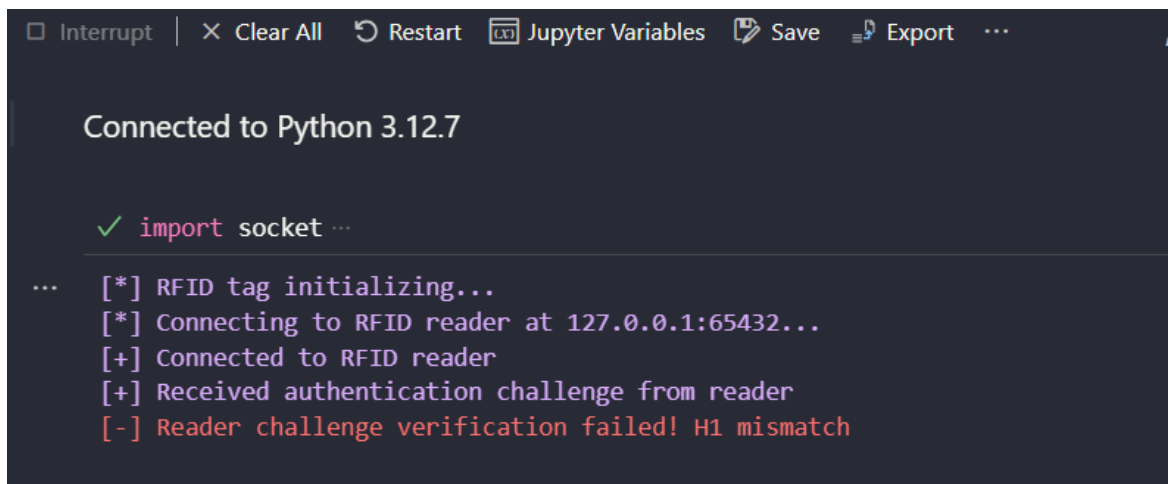
در بخش بعد باید سناریوهای failure رو بررسی کنیم.

4. سناریو Wrong PSK

در این بخش به اولین سناریو failure در این پروتکل میپردازیم و آن این است که آیا با تغییر PSK (pre shared key) آیا این مورد توسط پروتکل ما شناسایی می‌شود.

خب در ابتدا به شکل دستی میایم و مقدار PSK رو در فایل کلاینت به 810103099 تغییر میدهیم و به ترتیب ابتدا فایل سرور رو ران میکنیم تا سرور راه اندازی شد و سپس فایل کلاینت رو ران میگیریم.

خروجی رو میتونید در تصویر زیر مشاهده کنید:



```
Interrupt | X Clear All | Restart | Jupyter Variables | Save | Export | ...  
  
Connected to Python 3.12.7  
  
✓ import socket ...  
... [*] RFID tag initializing...  
[*] Connecting to RFID reader at 127.0.0.1:65432...  
[+] Connected to RFID reader  
[+] Received authentication challenge from reader  
[-] Reader challenge verification failed! H1 mismatch
```

شکل 32- خروجی در wrong PSK

خب همان طور که مشاهده میکنید این ارتباط fail شد به خاطر اینکه بعد از ارسال پیام از سرور به کلاینت و محاسبه $H1$ توسط کلاینت چون مقادیر کلید بین سرور و کلاینت یکی نبود در اینجا وریفای نشد و خطا داد.

$$H1 \neq H1_{prim}$$

این مورد به خاطر محاسبه $R1$ با مقادیر کلید متفاوت توسط تگ نیز میباشد.

1. سناریو Replay Attack

در این حمله فرد محاجم سعی دارد تا از اطلاعاتی که در session قبلی به دست آورده است دوباره استفاده کند که چون در پروتکل ما مقادیر $R1$ و $R2$ هر بار به شکل رندوم generate میشن این مورد شناسایی شده و با خطا رو به رو خواهد شد.

برای تست این مورد باید ابتدا کد سرور رو ران گرفته و سپس فایل Replay_Attack رو در یک ترمینال جدا اجرا کنید، تصویر مربوط به خروجی های این مورد در صفحه بعد قابل مشاهده است.

تصویر ترمینال خروجی سرور RFID reader:

```
Interrupt | X Clear All | Restart | Jupyter Variables | Save | Export | Python 3.12.7

Connected to Python 3.12.7

import socket ...

... [+] RFID reader listening on 127.0.0.1:65432
[*] Waiting for RFID tag connection...
[+] RFID tag connected from ('127.0.0.1', 13088)
[*] Generated random number R1: 4727448
[*] Sending authentication challenge to tag...
[*] Waiting for tag response...
[+] Received response from tag
[+] Tag authenticated successfully!
[*] Sending confirmation to tag...
[+] Mutual authentication completed successfully!
[*] Waiting for RFID tag connection...
[+] RFID tag connected from ('127.0.0.1', 13109)
[*] Generated random number R1: 7854734
[*] Sending authentication challenge to tag...
[*] Waiting for tag response...
[+] Received response from tag
[-] Tag authentication failed! H2 mismatch
[*] Waiting for RFID tag connection...
```

شکل 33- خروجی ترمینال سرور در Replay Attack

در ادامه خروجی ترمینال فایل Replay Attack هم قرار داده شده است:

```
Interrupt | X Clear All | Restart | Jupyter Variables | Save | Export | Python 3.12.7

Connected to Python 3.12.7

import socket ...

RFID REPLAY ATTACK SIMULATION

[*] Phase 1: Capturing legitimate communication...
[*] Connecting to RFID reader at 127.0.0.1:65432...
[+] Connected to RFID reader
[+] Captured M1, H1 from reader
M1: 8ed68fc87ff599e3da46953d398549c233ec5d4ad2b2ae9977f9980921cbbfe9
H1: 2bcf4b1a82cd02771cd167934129b4d4824809faf798aa2d2d9fcd0359d27429
[*] Sending legitimate M2, H2 to reader...
[+] Received H3 from reader
[*] Closing first connection
[*] Waiting 3 seconds before attempting replay attack...

[*] Phase 2: Performing replay attack...
[*] Connecting to RFID reader at 127.0.0.1:65432...
[+] Connected to RFID reader
[+] Received new M1', H1' from reader
New M1': 8ed68fc87ff599e3da46953d398549c233ec5d4ad2b2ae9977f9980921f447ff
New H1': f0dbbc3c1fea46352eea30741d3d7b14c3c52776601f7db5bbb55824b31e754e
[*] REPLAYING captured M2, H2 from previous session...
Replaying M2: 8ed68fc87ff599e3da46953d398549c233ec5d4ad2b2ae9977f99809213ffc3f
Replaying H2: 0fccc6da2d09934526a9f1f74ea88398d32d39b1947b455c206ac29c6c871502
[+] Received response from reader: Authentication Failed
[✓] Replay attack was DETECTED and PREVENTED!
[*] The protocol successfully prevented the replay attack because:
1. The reader generated a new random R1 for this session
2. H2 verification failed as it depends on the current session's R1
3. Replaying old M2,H2 with a new R1 will always fail verification
```

2. حمله MITM:

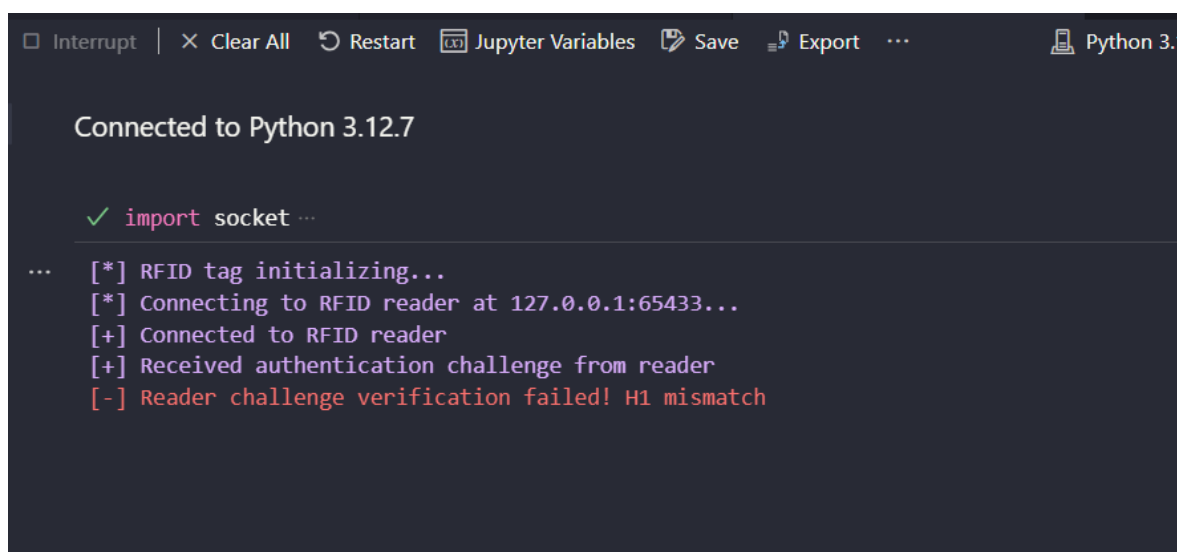
در این حمله یک Man in the middle وجود دارد که داره کانال ارتباطی بین کلاینت و سرور رو شنود میکند و سعی دارد تا پیام ارسالی را تغییر داده و به گسرنده ارسال کند حال هدف ما بررسی این است که ببینیم آیا پروتکل پیاده سازی شده ما در برابر این حمله مقاوم است یا نه. در واقع میخواهیم بررسی کنیم آیا تغییر پیام در سیستم شناسایی میشود یا خیر.

برای اینکار دو راه وجود دارد باید ابتدا یک فایل MITM.py بسازیم و قرار است در اینجا نقش proxy رو بازی کنه و کلاینت به جای اینکه پیام رو از سرور بگیره از MITM بگیره خب پس یا نیاز داریم یه MITM_client بسازیم یا اینکه port کلاینت رو با پورت MITM یکی بذاریم که این کار به شکل دستی انجام شده.

نکته: هر دو نوع این پیاده سازی ها انجام شده و فایل هاش به این گزارش ضمیمه شده است.

برای شروع تست ابتدا فایل سرور رو ران میگیریم و سپس فایل MITM_Attack و سپس فایل کلاینت با پورت عوض شده رو ران میگیریم:

خروجی ترمینال کلاینت بعد اجرا:



```
Connected to Python 3.12.7

✓ import socket ...

... [*] RFID tag initializing...
[*] Connecting to RFID reader at 127.0.0.1:65433...
[+] Connected to RFID reader
[+] Received authentication challenge from reader
[-] Reader challenge verification failed! H1 mismatch
```

شکل 34- خروجی ترمینال کلاینت با پورت 65433 در MITM attack

خروجی ترمینال MITM:

همان طور که میبینید بعد از دریافت پیام از MITM پیام دستکاری شده توسط کلاینت تشخیص داده شده و ارتباط توسط کلاینت قطع میشود.

```
Interrupt | X Clear All | Restart | Jupyter Variables | Save | Export | ... Python 3.12.7

Connected to Python 3.12.7

import socket ...

[!] Environment variables not set. You may need to modify client.py or set:
export RFID_HOST=127.0.0.1 RFID_PORT=65433
=====
RFID MAN-IN-THE-MIDDLE ATTACK SIMULATION
=====
[*] Starting MITM proxy...
[+] MITM proxy listening on 127.0.0.1:65433
[*] To test MITM attack, run client.py with HOST={MITM_HOST}, PORT={MITM_PORT}
[*] You can modify client.py temporarily to connect to the MITM proxy
[*] Or you can run: python client.py (after setting environment variables)
[*] Example: export RFID_HOST=127.0.0.1 RFID_PORT=65433 && python client.py
[+] Tag connected from ('127.0.0.1', 3340)
[+] Connected to reader at 127.0.0.1:65432
[+] Intercepted from reader: M1, H1
    Original M1: 8ed68fc87ff599e3da46953d398549c233ec5d4ad2b2ae9977f9980921cc91b2
    Original H1: e84537ee7be4683e13ac682fb06c6e4f73d4827c22c0f5e46bda60ea29c391dc
[!] MITM ATTACK: Modifying M1
    Modified M1: 8ed68fc87ff599e3da46953d398549c233ec5d4ad2b2ae9977f9980921cc914d
    Original H1: e84537ee7be4683e13ac682fb06c6e4f73d4827c22c0f5e46bda60ea29c391dc (unchanged)
[+] No response from tag (connection may have been closed)
```

شکل 35- خروجی MITM

خروجی ترمینال server:

```
Interrupt | X Clear All | Restart | Jupyter Variables | Save | Export | ...

Connected to Python 3.12.7

import socket ...

[+] RFID reader listening on 127.0.0.1:65432
[*] Waiting for RFID tag connection...
[+] RFID tag connected from ('127.0.0.1', 3341)
[*] Generated random number R1: 5180611
[*] Sending authentication challenge to tag...
[*] Waiting for tag response...
```

همان طور که مشاهده میکنید پروتکل ما در برابر این حمله نیز مقاوم است.

نکته: تمام تریس ها و لاگ ها در فایل های log به همراه کدها قابل مشاهده است.

((پایان))